MACHINE LEARNING

LAB RECORD

ITA0614 | EXPERIMENT 6

EXPERIMENT STATEMENT

Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

STUDENT NAME	REGISTER NUMBER
P H Devesh	**1911260236**
SUBJECT CODE	**SUBJECT NAME**
ITA0614	**Machine Learning**

Five program demonstrations • Executed Python examples • Verified output

SUB-PROGRAM 1

Naive Bayes from Scratch using Gaussian Likelihoods

Problem Statement / Program Title

Naive Bayes from Scratch using Gaussian Likelihoods.

Aim

Implement a simple Gaussian Naive Bayes classifier from scratch and predict the class of a new numeric sample.

Algorithm

1. Separate training rows by class and calculate each class prior probability.

2. For each feature within each class, calculate its mean and variance.

3. For a query, calculate the Gaussian likelihood for each feature and combine likelihoods with the class prior in log space.

4. Choose the class with the highest log posterior score.

5. Print class priors, the query, scores, and the predicted class.

Program

```
import math
from collections import defaultdict

data = [
    ([1.0, 2.0], "A"), ([1.2, 1.8], "A"), ([0.8, 1.6], "A"),
    ([4.0, 4.2], "B"), ([3.8, 4.0], "B"), ([4.2, 3.9], "B")
]
query = [1.1, 1.9]
classes = defaultdict(list)
for features, label in data:
    classes[label].append(features)

def gaussian_log_likelihood(x, mean, variance):
    variance = max(variance, 1e-9)
    return -0.5 * math.log(2 * math.pi * variance) - ((x-mean)**2)/(2*variance)

scores = {}
for label, rows in classes.items():
    score = math.log(len(rows) / len(data))
    for j in range(len(query)):
        values = [row[j] for row in rows]
        mean = sum(values) / len(values)
        variance = sum((v-mean)**2 for v in values) / len(values)
        score += gaussian_log_likelihood(query[j], mean, variance)
    scores[label] = score

print("Training samples:", len(data))
print("Class priors:", {c: round(len(v)/len(data), 3) for c, v in classes.items()})
print("Query:", query)
print("Log posterior scores:", {c: round(s, 3) for c, s in scores.items()})
print("Predicted class:", max(scores, key=scores.get))
```

Input / Dataset

Six labelled two-feature numeric samples split equally between classes A and B. The query [1.1, 1.9] is classified with Gaussian likelihoods.

Output

```
Training samples: 6
Class priors: {'A': 0.5, 'B': 0.5}
Query: [1.1, 1.9]
Log posterior scores: {'A': 0.718, 'B': -302.61}
Predicted class: A
```

Result

The Gaussian Naive Bayes implementation calculated class log scores and classified the query as class A.

SUB-PROGRAM 2

Naive Bayes Classification on the Iris Dataset

Problem Statement / Program Title

Naive Bayes Classification on the Iris Dataset.

Aim

Train a Gaussian Naive Bayes model on Iris data and evaluate it with a confusion matrix and accuracy.

Algorithm

1. Load the Iris dataset with four numeric flower measurements and three species labels.
2. Split the data into stratified training and test sets.
3. Fit Gaussian Naive Bayes using the training records.
4. Predict labels for the held-out test records.
5. Calculate test accuracy and the confusion matrix.
6. Predict the species for one example flower.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25,
    random_state=42, stratify=iris.target
)
model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
sample = [[5.1, 3.5, 1.4, 0.2]]

print("Dataset: Iris")
print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
print("Test accuracy:", f"{accuracy_score(y_test, pred)*100:.2f}%")
print("Confusion matrix (rows=true, columns=predicted):")
print(confusion_matrix(y_test, pred))
print("Sample:", sample[0])
print("Predicted species:", iris.target_names[model.predict(sample)[0]])
```

Input / Dataset

Iris dataset: 150 samples, four numeric features, and three species. A stratified 75/25 split produces 112 training and 38 test samples.

Output

```
Dataset: Iris
Training samples: 112
Test samples: 38
Test accuracy: 92.11%
Confusion matrix (rows=true, columns=predicted):
[[12  0  0]
 [ 0 12  1]
 [ 0  2 11]]
Sample: [5.1, 3.5, 1.4, 0.2]
Predicted species: setosa
```

Result

Gaussian Naive Bayes achieved 92.11% accuracy on the held-out Iris test set; the confusion matrix shows the per-class prediction counts.

SUB-PROGRAM 3

Multinomial Naive Bayes for Text Classification

Problem Statement / Program Title

Multinomial Naive Bayes for Text Classification.

Aim

Use Multinomial Naive Bayes to classify short text messages into predefined categories.

Algorithm

1. Create a small labelled collection of example messages.
2. Split the messages into training and test subsets.
3. Convert text into word-count vectors using CountVectorizer.
4. Train a Multinomial Naive Bayes classifier on the vectors.
5. Predict the test labels and calculate test accuracy.
6. Classify a new message and display its predicted category.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

texts = [
    "team won the cricket match", "player scored a goal",
    "the match starts today", "coach trained the team",
    "new phone has a bright screen", "laptop has fast processor",
    "the phone battery lasts long", "computer software update",
    "cricket team won the final", "goal scored by player",
    "new laptop and computer", "phone screen and battery"
]
labels = [
    "sports","sports","sports","sports",
    "technology","technology","technology","technology",
    "sports","sports","technology","technology"
]
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, random_state=5, stratify=labels
)
vectorizer = CountVectorizer()
Xtr = vectorizer.fit_transform(X_train)
Xte = vectorizer.transform(X_test)
model = MultinomialNB()
model.fit(Xtr, y_train)
pred = model.predict(Xte)
new_text = ["cricket player won the match"]
new_vector = vectorizer.transform(new_text)

print("Training messages:", len(X_train))
print("Test messages:", len(X_test))
print("Test accuracy:", f"{accuracy_score(y_test, pred)*100:.1f}%")
print("Actual test labels:", list(y_test))
print("Predicted test labels:", list(pred))
print("New message:", new_text[0])
print("Predicted category:", model.predict(new_vector)[0])
```

Input / Dataset

Twelve short labelled messages across sports and technology categories. The fixed stratified split contains 9 training and 3 test messages; this tiny set is illustrative.

Output

```
Training messages: 9
Test messages: 3
Test accuracy: 100.0%
Actual test labels: ['technology', 'sports', 'technology']
Predicted test labels: [np.str_('technology'), np.str_('sports'), np.str_('technology')]
New message: cricket player won the match
Predicted category: sports
```

Result

Multinomial Naive Bayes correctly classified all three test messages in this small split and classified the new message as sports. The tiny test set makes the accuracy uncertain.

SUB-PROGRAM 4

Bernoulli Naive Bayes for Binary Features

Problem Statement / Program Title

Bernoulli Naive Bayes for Binary Features.

Aim

Demonstrate Bernoulli Naive Bayes for classification when features indicate the presence or absence of events.

Algorithm

1. Define a small binary-feature dataset with two class labels.
2. Split examples into training and test records.
3. Train Bernoulli Naive Bayes with binarized feature inputs.
4. Predict the test labels and calculate accuracy.
5. Predict the class of a new binary feature vector.
6. Print the dataset dimensions, predictions, and accuracy.

Program

```
from sklearn.naive_bayes import BernoulliNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X = [
    [1,1,0], [1,0,0], [1,1,1], [1,0,1],
    [0,1,1], [0,0,1], [0,1,0], [0,0,0],
    [1,1,0], [0,1,1], [1,0,0], [0,0,1]
]
y = ["A","A","A","A","B","B","B","B","A","B","A","B"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=4, stratify=y
)
model = BernoulliNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
new_sample = [[1,1,0]]

print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
print("Test accuracy:", f"{accuracy_score(y_test, pred)*100:.1f}%")
print("Actual test labels:", list(y_test))
print("Predicted test labels:", list(pred))
print("New binary sample:", new_sample[0])
print("Predicted class:", model.predict(new_sample)[0])
```

Input / Dataset

Twelve examples with three binary features and class labels A/B. A fixed stratified split contains 9 training and 3 test examples.

Output

```
Training samples: 9
Test samples: 3
Test accuracy: 100.0%
Actual test labels: ['A', 'A', 'B']
Predicted test labels: [np.str_('A'), np.str_('A'), np.str_('B')]
New binary sample: [1, 1, 0]
Predicted class: A
```

Result

Bernoulli Naive Bayes achieved 100.0% accuracy on the three test samples in this split and predicted class A for the new binary vector; the sample is too small for broad conclusions.

SUB-PROGRAM 5

Bank Loan Approval Prediction using Naive Bayes

Problem Statement / Program Title

Bank Loan Approval Prediction using Naive Bayes.

Aim

Build a small illustrative Naive Bayes classifier to predict loan approval from categorical applicant features.

Algorithm

1. Prepare illustrative records with income, employment, credit history, and loan approval label.
2. Encode the categorical features as numeric categories.
3. Train a categorical Naive Bayes model on the example records.
4. Predict the approval label for a new applicant profile.
5. Display the sample, predicted decision, and training accuracy.
6. Treat this synthetic exercise as a learning example, not a real credit decision system.

Program

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import accuracy_score

X_text = [
    ["High","Stable","Good"], ["Medium","Stable","Good"],
    ["High","Unstable","Good"], ["Low","Stable","Poor"],
    ["Low","Unstable","Poor"], ["Medium","Unstable","Poor"],
    ["High","Stable","Good"], ["Medium","Stable","Poor"],
    ["Low","Stable","Good"], ["High","Unstable","Poor"],
    ["Medium","Stable","Good"], ["Low","Unstable","Poor"]
]
y_text = ["Approved","Approved","Approved","Rejected","Rejected","Rejected",
          "Approved","Rejected","Rejected","Rejected","Approved","Rejected"]
encoder = OrdinalEncoder()
X = encoder.fit_transform(X_text).astype(int)
target_encoder = LabelEncoder()
y = target_encoder.fit_transform(y_text)

model = CategoricalNB()
model.fit(X, y)
pred = model.predict(X)
new_applicant = [["High","Stable","Good"]]
prediction = target_encoder.inverse_transform(
    model.predict(encoder.transform(new_applicant).astype(int))
)[0]

print("Dataset type: synthetic illustrative applicant records")
print("Training records:", len(X))
print("Training accuracy:", f"{accuracy_score(y, pred)*100:.1f}%")
print("New applicant [Income=High, Employment=Stable, Credit=Good]")
print("Predicted decision:", prediction)
```

Input / Dataset

Twelve synthetic applicant records using income, employment, and credit-history categories. This is a classroom illustration, not a real lending dataset.

Output

```
Dataset type: synthetic illustrative applicant records
Training records: 12
Training accuracy: 91.7%
New applicant [Income=High, Employment=Stable, Credit=Good]
Predicted decision: Approved
```

Result

Categorical Naive Bayes classified the illustrative applicant profile as Approved with 91.7% training accuracy. This is not suitable for real credit decisions.